# Fundamentos de Aprendizagem Automática 2025/2026

## Cátia Pesquita + Diogo F. Soares + Helena Aidos + Joana C. Santos, 2025/2026

# Practical 2.2 - K-Nearest Neighbors (KNN)

## 0. Introduction

This tutorial introduces **classification and regression using the K-Nearest Neighbors (KNN) algorithm**, one of the simplest and most intuitive machine learning methods.

KNN is a supervised learning algorithm that makes predictions based on the similarity between data points. Instead of learning an explicit mathematical model during training, KNN stores the training data and makes predictions by finding the k most similar examples (neighbors) to a new observation.

Because of its simplicity and interpretability, KNN is often used as:

- A baseline model in machine learning experiments
- A tool to build intuition about distance-based learning methods
- A practical method for small and medium-sized datasets

**The tutorial has 2 main parts:**

1. K-Nearest Neighbours
    a. K-Nearest neighbours for classification
    b. K-Nearest neighbours for regression
    c. Improving the model with Kernels
2. Scaling Data

Scikit learn has two powerful KNN implementations for classification and regression, respectively.

Both the regression and classification implementations allow to select the number of Neighbours and different instance weighting based on distance. They also allow for using distance modification functions for more complex kernels.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor,
KNeighborsClassifier
from sklearn.metrics import mean_squared_error, accuracy_score
```

# 1. K-Nearest Neighbours

## 1.1 KNN for classification

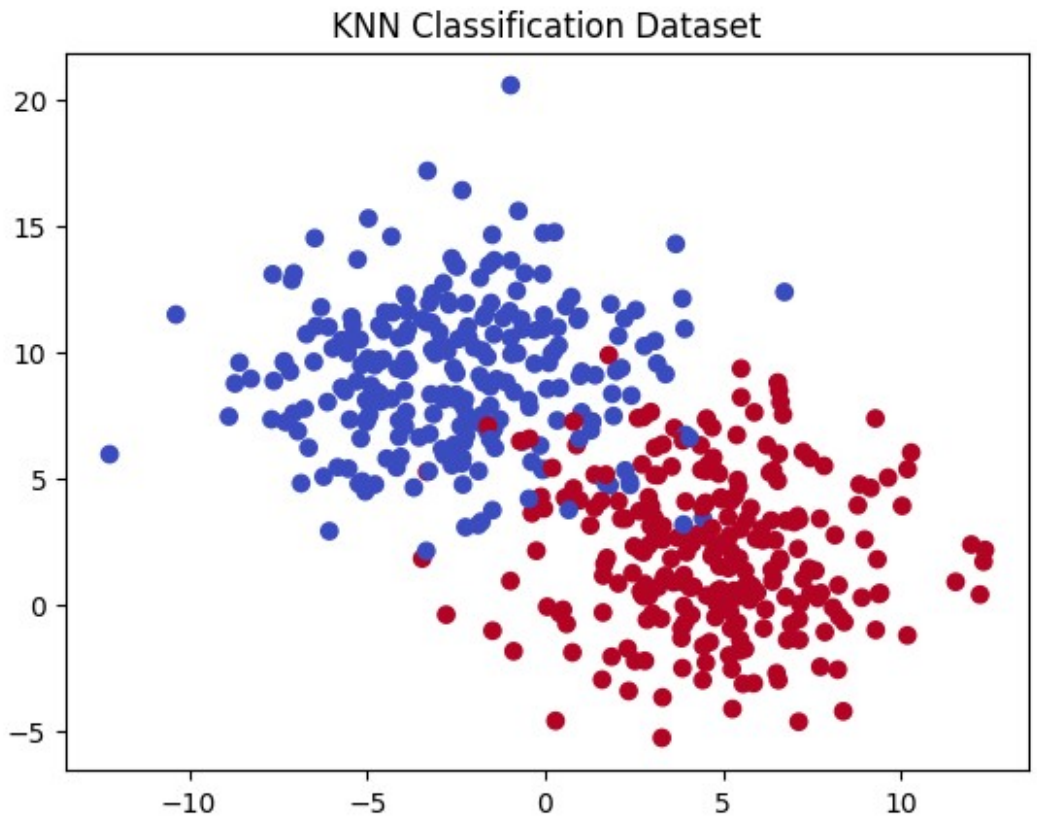
We will now use `KNeighborsClassifier` for classification (https://scikit-learn.org/stable/modules/neighbors.html#nearest-neighbors-classification).

We are going to start with synthetic data for visualizing the model.

```
#Generating 2 clouds of points
from sklearn.datasets import make_blobs

X, y = make_blobs(
    n_samples=500,
    centers=2,
    random_state=42,
    cluster_std=3
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

knn_clf = KNeighborsClassifier(n_neighbors=5)

knn_clf.fit(X_train, y_train)

y_pred = knn_clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

```
Accuracy: 0.99
```

```
plt.scatter(X[:,0], X[:,1], c=y, cmap="coolwarm")
plt.title("KNN Classification Dataset")
plt.show()
```

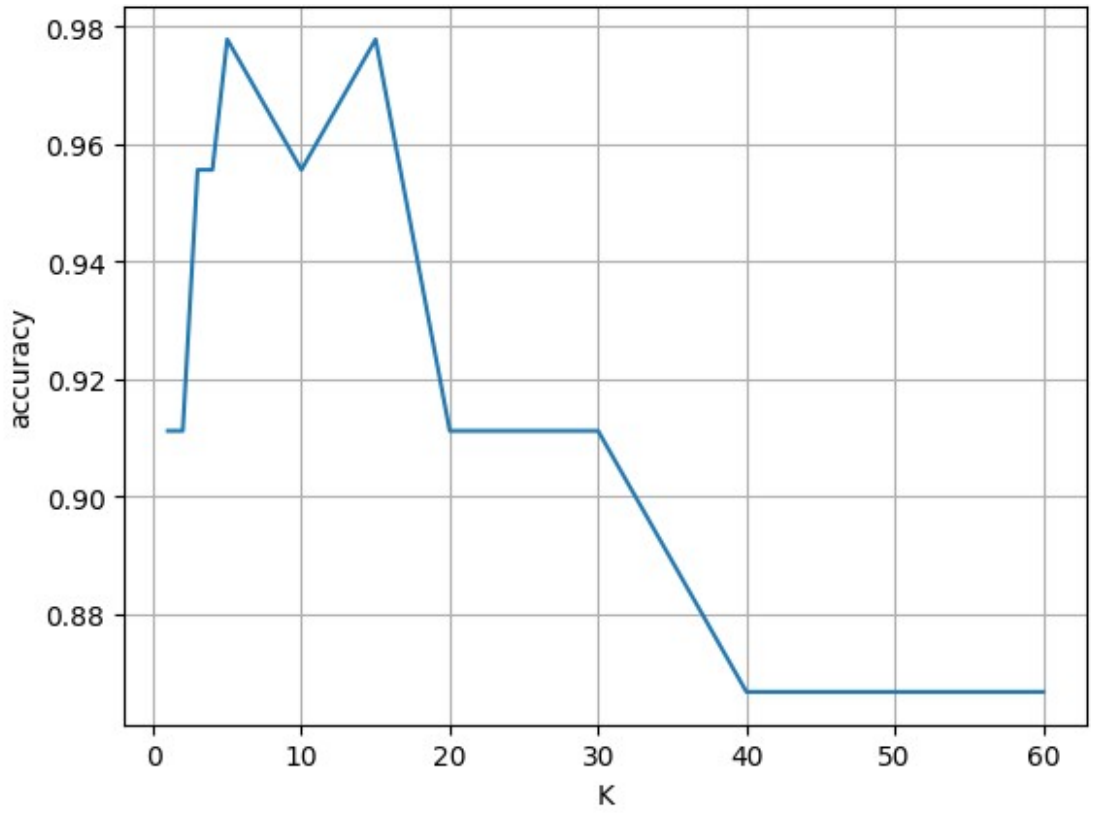



Now, let's recall the iris dataset!

```
from sklearn.datasets import load_iris
X,y=load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=123)
```

Now, you are probably wondering: **How to choose the best value K?** Let's plot how the accuracy is affected by the number of neighbours K

```
ks=np.array([1,2,3,4,5,10, 15, 20, 30, 40, 60])
accs=np.zeros(ks.shape[0])

for i,k in enumerate(ks):
    knn_clf = KNeighborsClassifier(n_neighbors=k).fit(X_train,
y_train)
    preds=knn_clf.predict(X_test)
    accs[i]=accuracy_score(y_test, preds)

plt.plot(ks, accs)
plt.grid()
plt.xlabel("K")
plt.ylabel("accuracy")
plt.show()
```




# 1.2 KNN for regression

## 1.1. Distance weighting

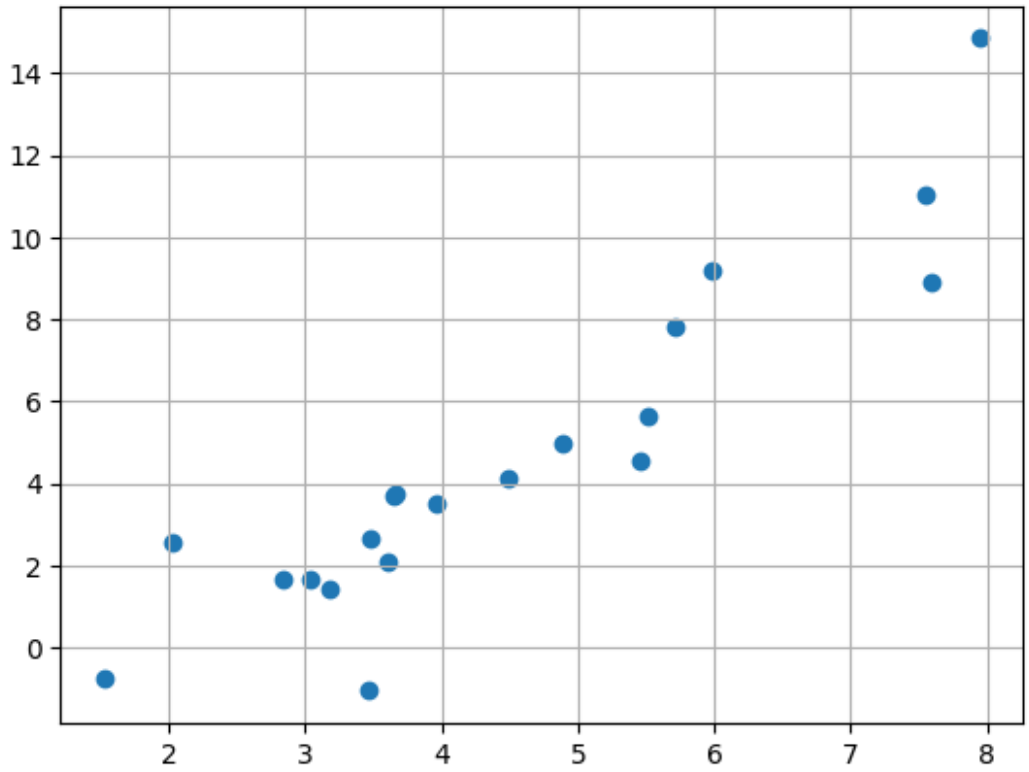
By default KNN regression or classification considers each neighbour as contributing equally to the final result, disregarding any effect of the distance. Yet it may be convenient to weight each neighbour according to the inverse of the distance.

$$w_i = \frac{1}{d_i}$$

Larger distances will have a smaller impact on the inferred value.

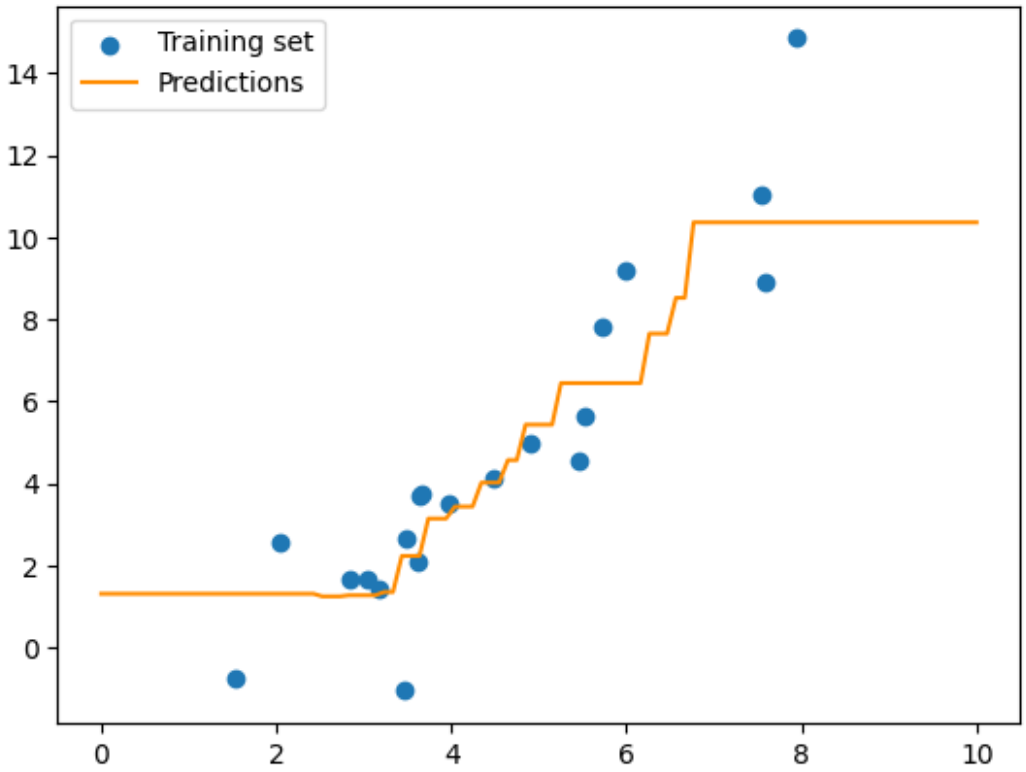
To visualise the effect of the weight function in regression we are going to use a very simple data problem with 1 feature. The testing set contains 100 X values ranging from 0.0 to 10.0

```
import pickle
X_train, y_train, X_test, y_test = pickle.load(open("data.pickle",
"rb"))
plt.scatter(X_train[:,0], y_train)
plt.grid()
plt.show()
```



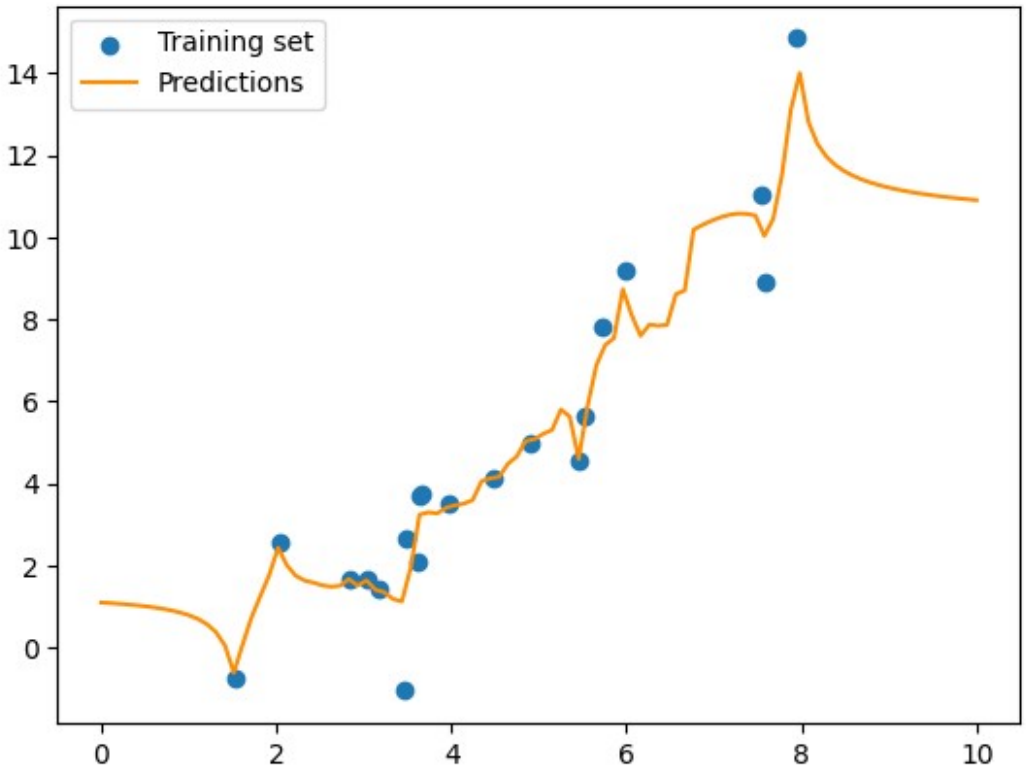

The unweighted KNN regressor works in steps --- as long as the neighbours do not change, the prediction will also not change.

```
knn = KNeighborsRegressor(n_neighbors=5)
knn.fit(X_train, y_train)
preds = knn.predict(X_test)
plt.scatter(X_train,y_train, label="Training set")
plt.plot(X_test, preds, color="darkorange", label="Predictions")
plt.legend()
plt.show()
```




With inverse distance weighting the impact of each nearby point can be seen a spike and the step effect mostly disapears.

```
knn = KNeighborsRegressor(n_neighbors=5, weights="distance")
knn.fit(X_train, y_train)
preds = knn.predict(X_test)
plt.scatter(X_train,y_train, label="Training set")
plt.plot(X_test, preds, color="darkorange", label="Predictions")
plt.legend()
plt.show()
```



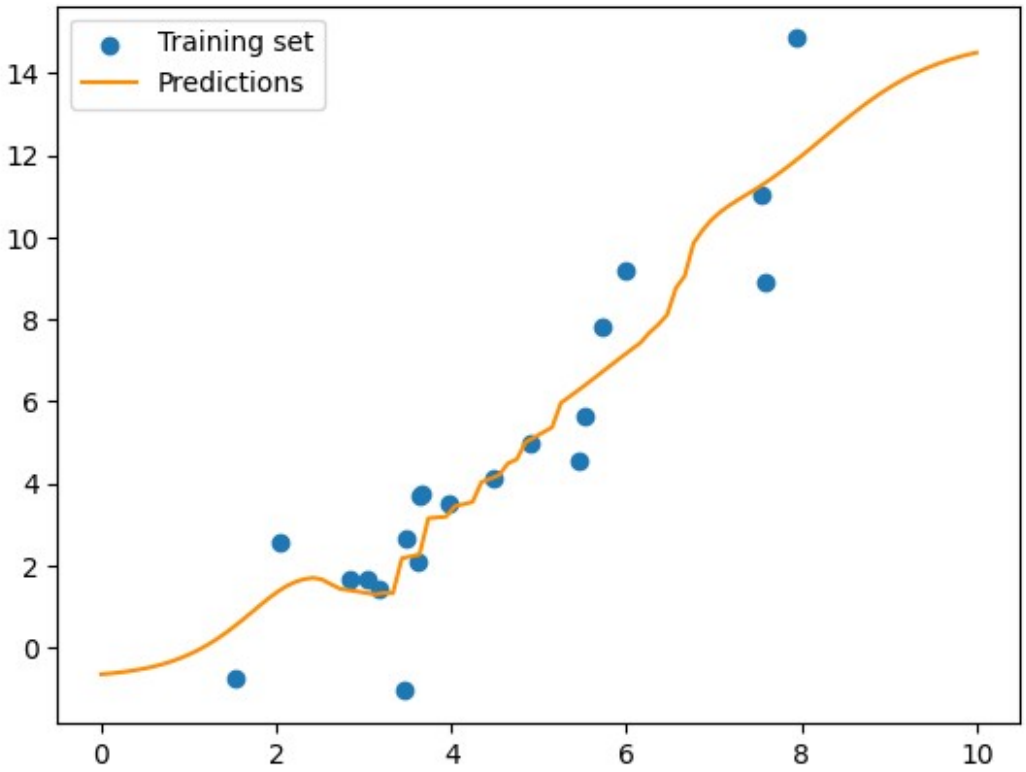

## 1.4 Kernel Weighting

Inverse distance weighting can be replaced by any function, for example a sigmoid function. Replacing the distance with a sigmoid:

$$w = e^{-\frac{d^2}{kw}}$$

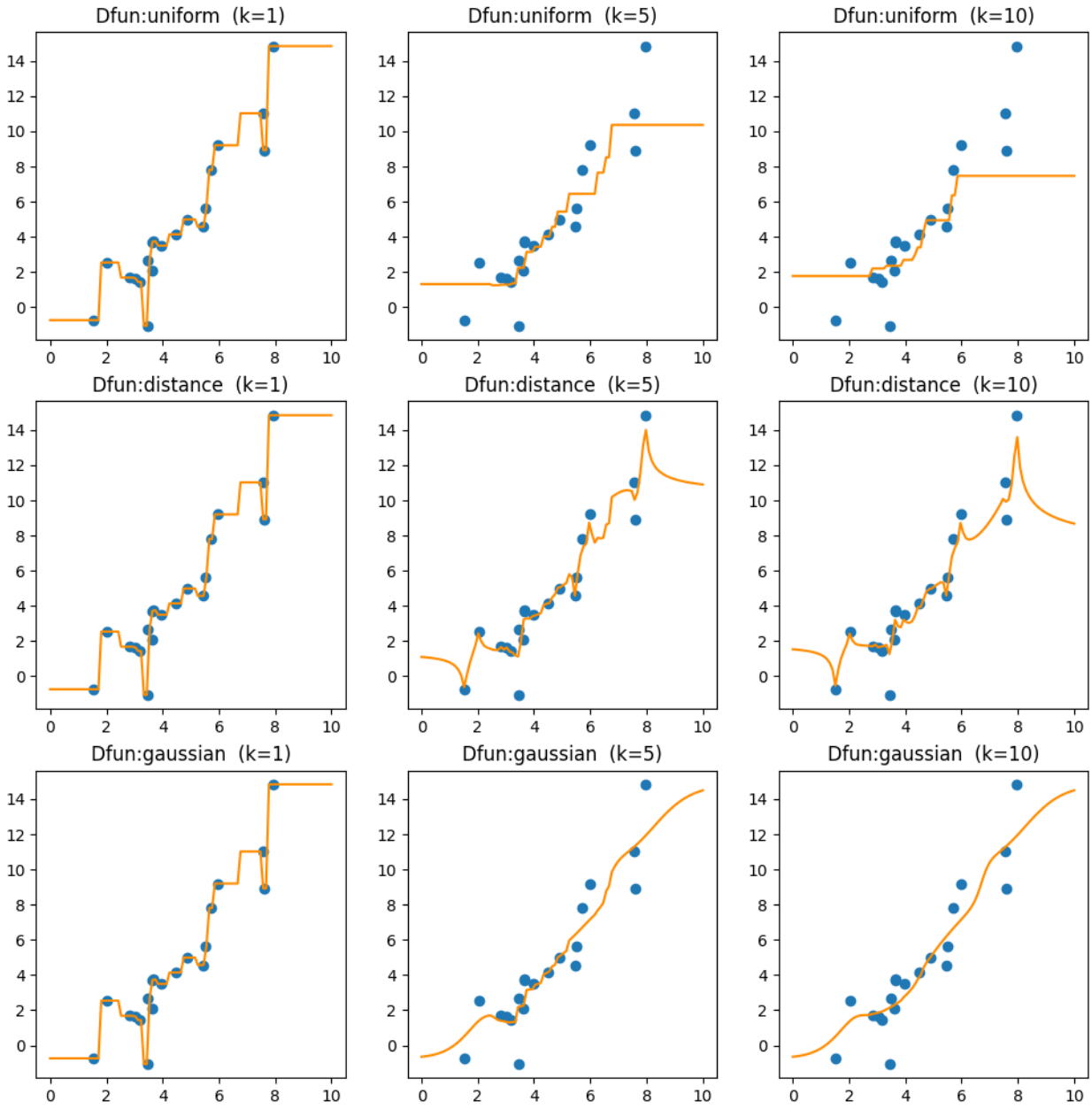
This function requires one further parameter, `kw`, a kernel weight function that can be fit for whatever purpose. This softer weighting will create a better interpolation.

```
def gaussian(dsts):
    kernel_width = .5
    weights = np.exp(-(dsts**2)/kernel_width)
    return weights

knn = KNeighborsRegressor(n_neighbors=5, weights=gaussian)
knn.fit(X_train, y_train)
preds = knn.predict(X_test)
plt.scatter(X_train,y_train, label="Training set")
plt.plot(X_test, preds, color="darkorange", label="Predictions")
plt.legend()
plt.show()
```




Let's see the impact of K vs weighting functions in a single plot combination

```
fig, axs = plt.subplots(3, 3, figsize=(12,12))
for i, wfun in enumerate( ["uniform", "distance", gaussian]):
    for j, k in enumerate([1,5,10]):
        knn = KNeighborsRegressor(n_neighbors=k, weights=wfun)
#weights="distance")
        knn.fit(X_train, y_train)
        preds = knn.predict(X_test)
        s= wfun if wfun!=gaussian else "gaussian"
        axs[i,j].set_title("Dfun:%s  (k=%d)" %(s, k))
        axs[i,j].scatter(X_train,y_train)
        axs[i,j].plot(X_test, preds, color="darkorange")
```




## 1.5 Fitting KNN Regression to an actual data set Scikit Learn

Let's try KNN Regression with the Diabetes dataset with K=5 and do a full regression analysis.

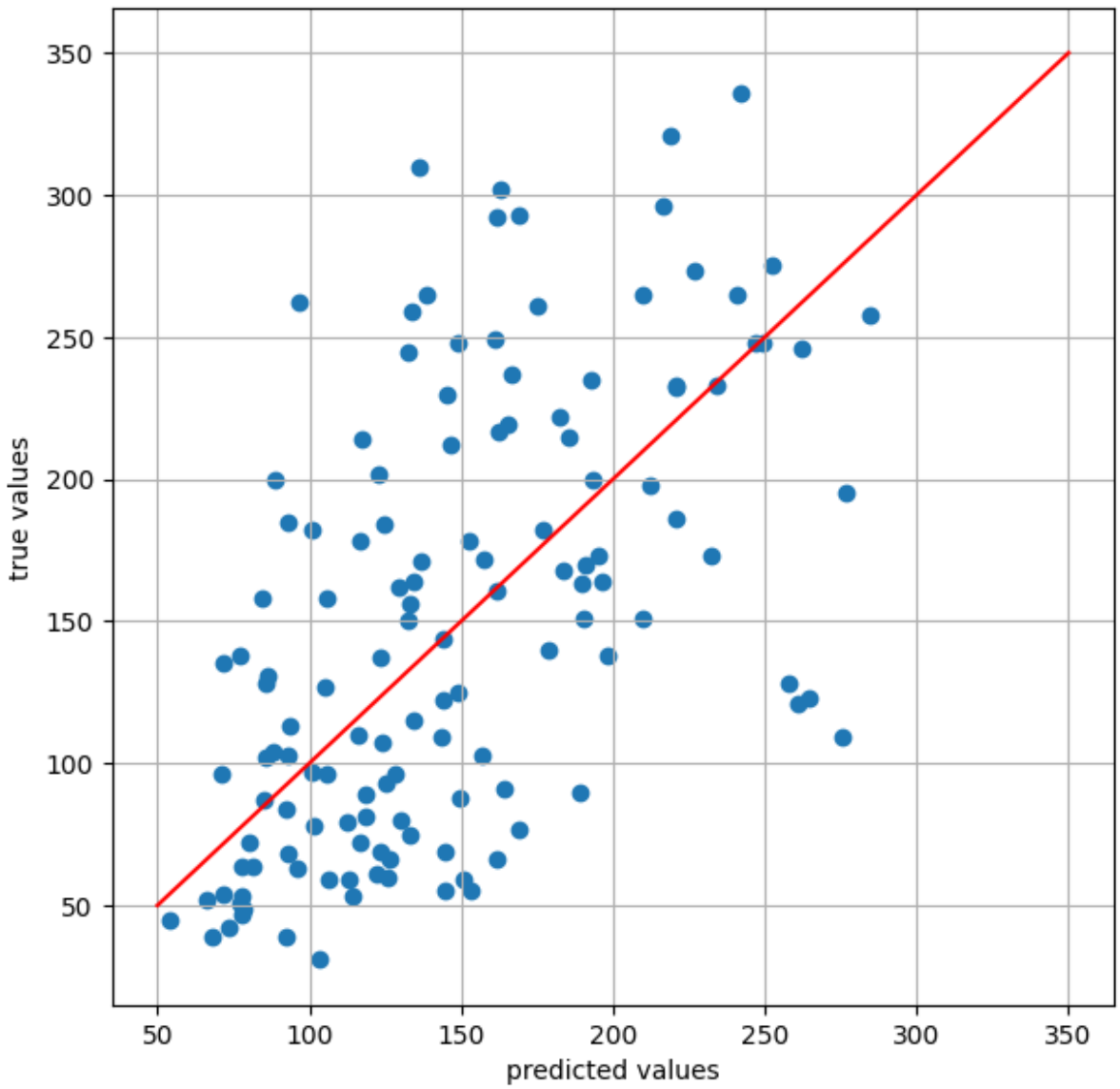
```
from sklearn.datasets import load_diabetes
X,y=load_diabetes(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=123)
X_train.shape
```

```
(309, 10)
```

This will fit a KNN with K=3 and no distance weighting.

```
from sklearn.metrics import explained_variance_score,
root_mean_squared_error, max_error, mean_absolute_error
from scipy.stats import pearsonr


knn = KNeighborsRegressor(n_neighbors=3 )
knn.fit(X_train, y_train)
preds = knn.predict(X_test)

preds=knn.predict(X_test)


print("The rmse is: ", root_mean_squared_error(y_test, preds))
corr, pval=pearsonr(y_test, preds)
print("The Correlation Score is is: %6.4f (p-value=%e)\n"%(corr,pval))
print("The Maximum Error is is: ", max_error(y_test, preds))
print("The Mean Absolute Error is: ", mean_absolute_error(y_test,
preds))


plt.figure(figsize=(7,7))
plt.scatter(preds, y_test)
plt.plot((50, 350), (50, 350), c="r")
plt.xlabel("predicted values")
plt.ylabel("true values")
plt.grid()
plt.show()
```

```
The rmse is:  64.4442535934719
The Correlation Score is is: 0.5686 (p-value=9.340392e-13)

The Maximum Error is is:  173.66666666666666
The Mean Absolute Error is:  50.954887218045116
```

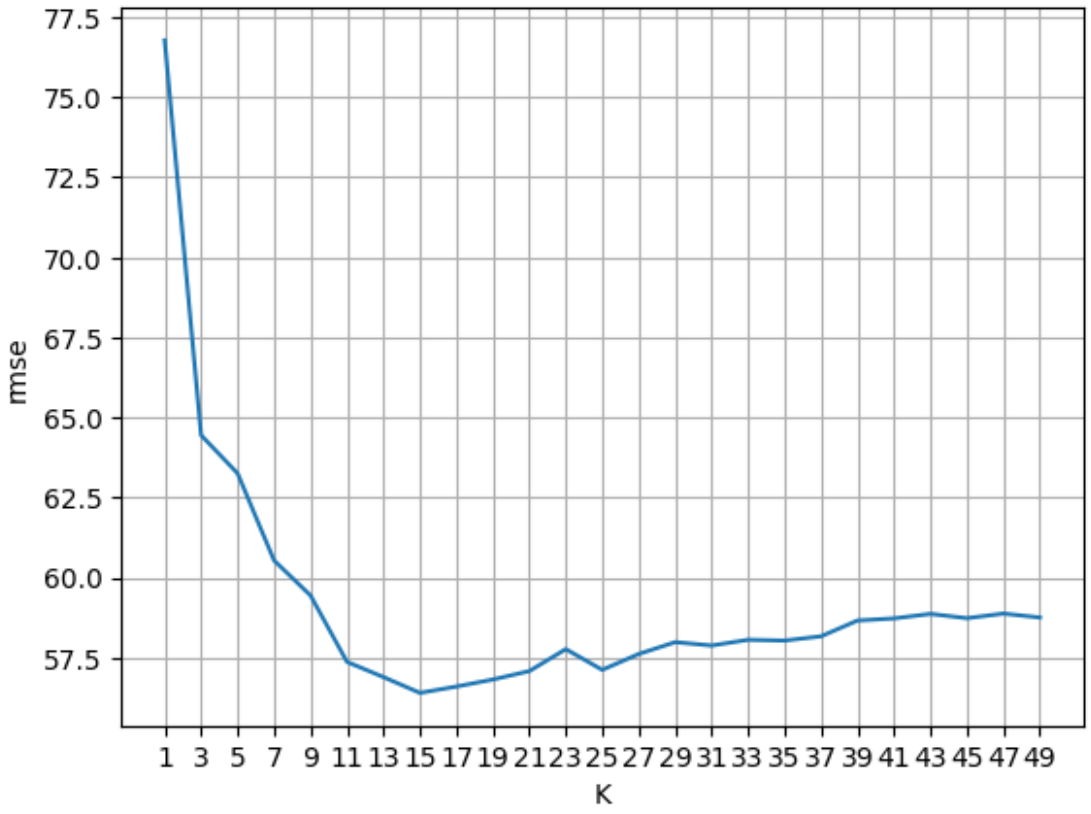



Now let's change K to investigate its impact on the RMSE

```
ks=np.arange(1,50,2)
print(ks)
rmses=np.zeros(ks.shape[0])
print(len(X_train))
for i,k in enumerate(ks):
    knn = KNeighborsRegressor(n_neighbors=k)
    knn.fit(X_train, y_train)
    preds = knn.predict(X_test)
    rmses[i]=root_mean_squared_error(y_test, preds)

print(rmses)
plt.plot(ks, rmses)
plt.xticks(ks)
```

```
plt.grid()
plt.ylabel("rmse")
plt.xlabel("K")
plt.show()
```

```
[ 1  3  5  7  9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45
47
 49]
309
[76.78032329 64.44425359 63.25342512 60.5346911  59.44040461
57.36576111
 56.89173281 56.39816352 56.59612249 56.81597987 57.07846921
57.76478339
 57.11735299 57.6126523  57.98256571 57.87746091 58.05571004
58.03186877
 58.16485667 58.65832838 58.72458365 58.86566634 58.73128634
58.87487426
 58.75332085]
```




## 1.6 Exercise

1. Considering the results above, what is the best K?
2. How confident are you in this result? What would you do to check it out?

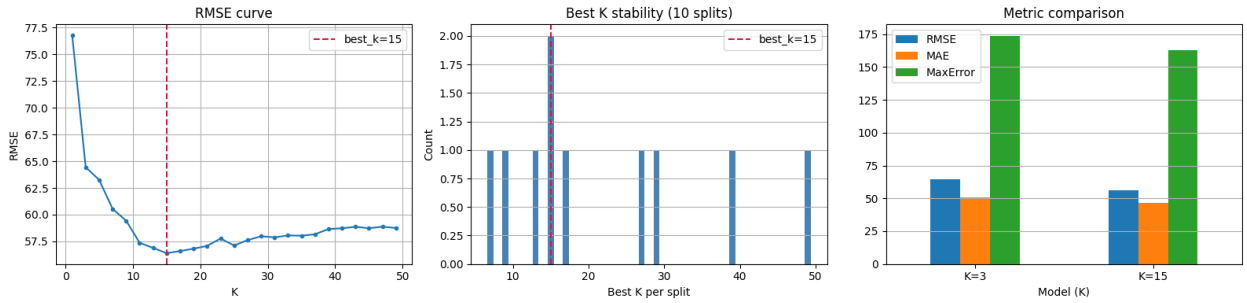
3. With this "optimal K", compute the regression metrics and compare them to the original estimate

```
# 1) Best K from the RMSE curve above
best_k = int(ks[np.argmin(rmses)])
print("1) Best K:", best_k)

# 2) Confidence using only methods already used above (different
train/test splits)
best_ks = []
for rs in range(10):
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
random_state=rs)
    tmp_rmses = []
    for k_val in ks:
        model = KNeighborsRegressor(n_neighbors=int(k_val))
        pred = model.fit(Xtr, ytr).predict(Xte)
        tmp_rmses.append(root_mean_squared_error(yte, pred))
    best_ks.append(int(ks[np.argmin(tmp_rmses)]))

print("Best K across 10 splits:", best_ks)
"Confidence is higher if the same/similar K appears often."

# 3) Compare metrics: original K=3 vs optimal K
rows = []
for label, k in [("K=3", 3), (f"K={best_k}", best_k)]:
    pred = KNeighborsRegressor(n_neighbors=k).fit(X_train,
y_train).predict(X_test)
    corr, _ = pearsonr(y_test, pred)
    rows.append([label, root_mean_squared_error(y_test, pred),
mean_absolute_error(y_test, pred), corr, max_error(y_test, pred)])

"3) Metrics comparison (lower RMSE/MAE/MaxError is better; higher Corr
is better):"
display(pd.DataFrame(rows, columns=["Model (K)", "RMSE", "MAE",
"Corr", "MaxError"]))

###### NAO
###### É
###### SÓ PARA VER


# Visual summary: RMSE curve + stability of best K across splits +
metric comparison
fig, axs = plt.subplots(1, 3, figsize=(16, 4))

# (A) RMSE vs K (test split used above)
axs[0].plot(ks, rmses, marker="o", ms=3)
axs[0].axvline(best_k, color="crimson", linestyle="--",
label=f"best_k={best_k}")
axs[0].set_xlabel("K")
```

```
axs[0].set_ylabel("RMSE")
axs[0].set_title("RMSE curve")
axs[0].grid(True)
axs[0].legend()

# (B) Best K frequency across random splits (confidence view)
u, c = np.unique(best_ks, return_counts=True)
axs[1].bar(u, c, color="steelblue")
axs[1].axvline(best_k, color="crimson", linestyle="--",
label=f"best_k={best_k}")
axs[1].set_xlabel("Best K per split")
axs[1].set_ylabel("Count")
axs[1].set_title("Best K stability (10 splits)")
axs[1].grid(True, axis="y")
axs[1].legend()

# (C) Metrics comparison (excluding Corr for same scale)
cmp_df = pd.DataFrame(rows, columns=["Model (K)", "RMSE", "MAE",
"Corr", "MaxError"])
cmp_df.plot(x="Model (K)", y=["RMSE", "MAE", "MaxError"], kind="bar",
ax=axs[2], rot=0)
axs[2].set_title("Metric comparison")
axs[2].grid(True, axis="y")

plt.tight_layout()
plt.show()
```

```
1) Best K: 15
Best K across 10 splits: [27, 9, 39, 29, 17, 7, 15, 15, 49, 13]
```

| | Model (K) | RMSE | MAE | Corr | MaxError |
|---|---|---|---|---|---|
| 0 | K=3 | 64.444254 | 50.954887 | 0.568629 | 173.666667 |
| 1 | K=15 | 56.398164 | 46.301253 | 0.696608 | 163.000000 |

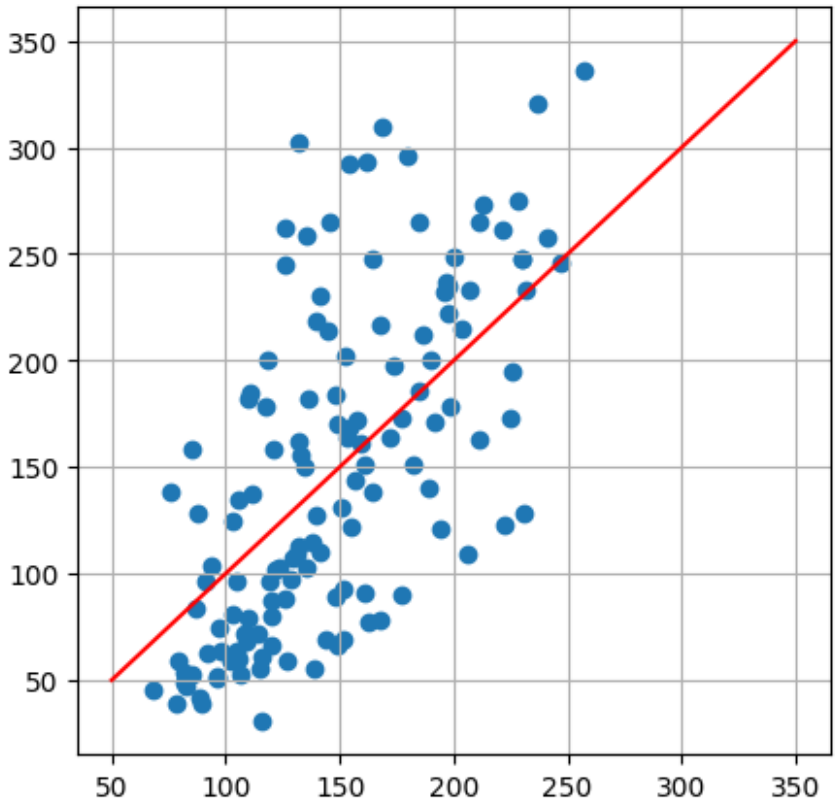



Now testing with inverse distance weights

```
knn = KNeighborsRegressor(n_neighbors=10, weights="distance")
knn.fit(X_train, y_train)
preds_w = knn.predict(X_test)
```

```
def present_reg_statistics(y_test, preds):
    print("The rmse is: ", root_mean_squared_error(y_test, preds))
    corr, pval=pearsonr(y_test, preds)
    print("The Correlation Score is is: %6.4f (p-value=%e)\n"%
(corr,pval))
    print("The Maximum Error is is: ", max_error(y_test, preds))
    print("The Mean Absolute Error is: ", mean_absolute_error(y_test,
preds))
    plt.figure(figsize=(5,5))
    plt.scatter(preds, y_test)
    plt.plot((50, 350), (50, 350), c="r")
    plt.grid()
    plt.show()

present_reg_statistics(y_test, preds_w)
```

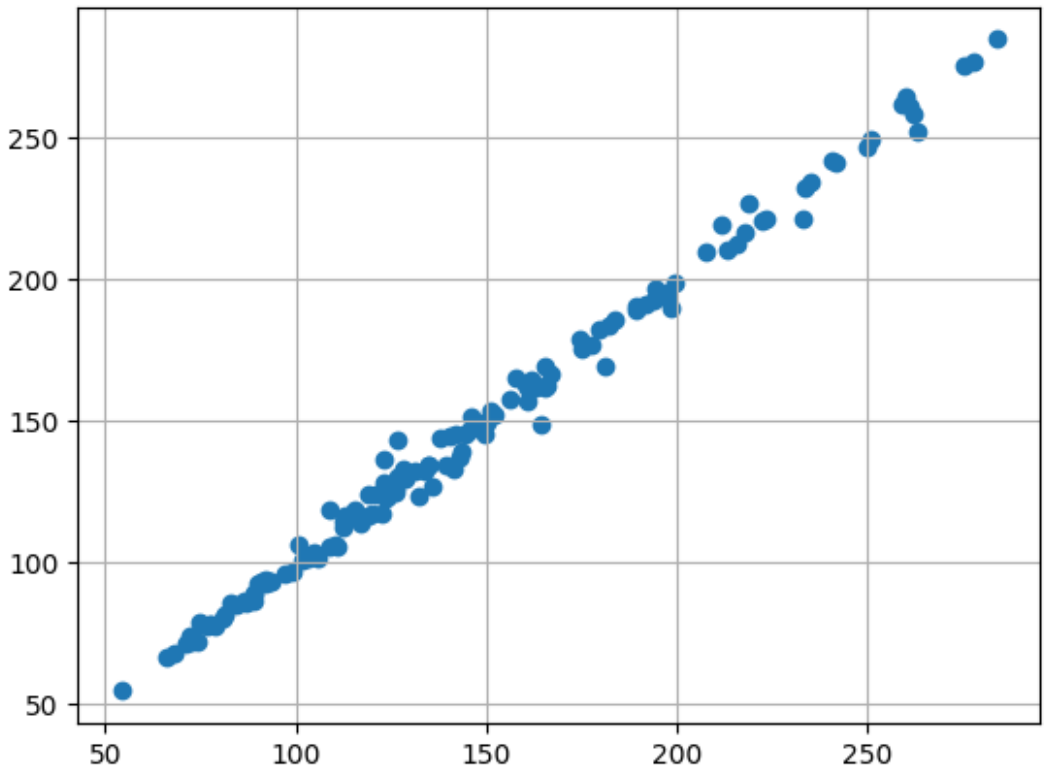
```
The rmse is:  57.930137146579185
The Correlation Score is is: 0.6655 (p-value=2.371272e-18)

The Maximum Error is is:  169.93473869767072
The Mean Absolute Error is:  46.99856243744194
```




We can compare the actual impact of distance weighting, which for this specific example is not very expressive

First let's compare the predictions with and without distance weighting

```
knn = KNeighborsRegressor(n_neighbors=3)
knn.fit(X_train, y_train)
preds = knn.predict(X_test)

knn = KNeighborsRegressor(n_neighbors=3, weights="distance")
knn.fit(X_train, y_train)
preds_w = knn.predict(X_test)
plt.scatter(preds_w, preds)
plt.grid()
plt.show()
```

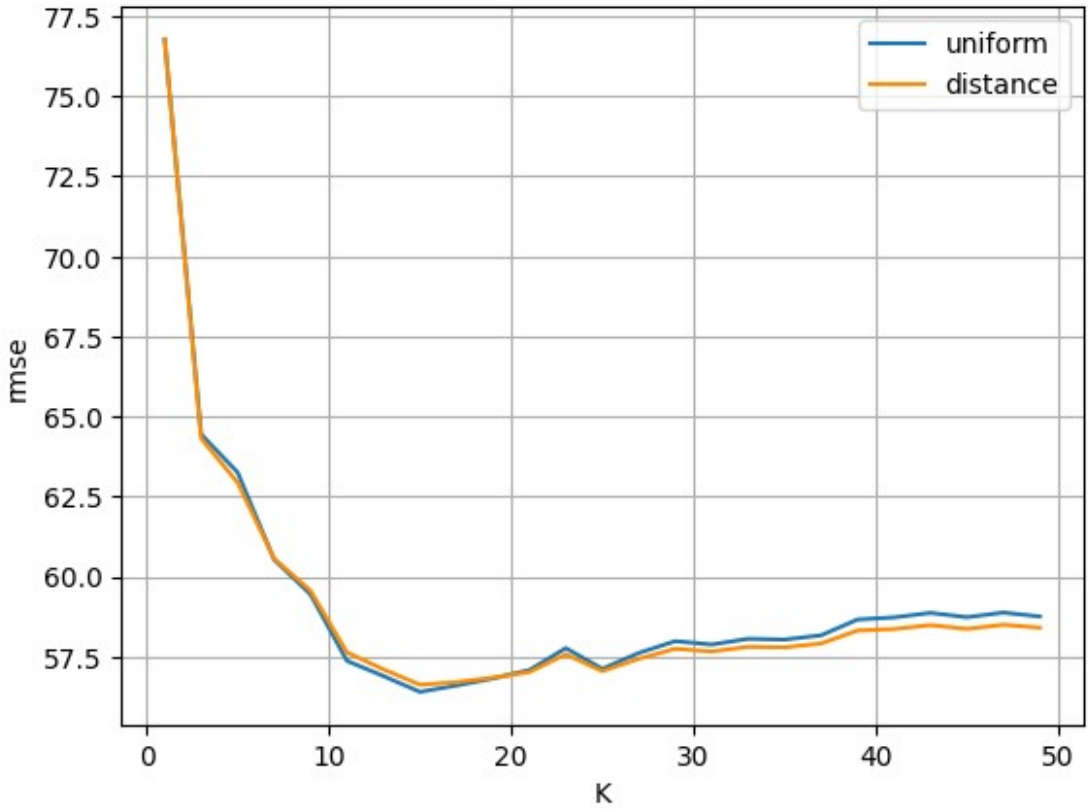



We can check that for this specific problem the differences are not too big

How do they fare for different K?

```
ks=np.arange(1,50,2)
rmses_w=np.zeros(ks.shape[0])
for i,k in enumerate(ks):
    knn = KNeighborsRegressor(n_neighbors=k, weights="distance")
    knn.fit(X_train, y_train)
    preds = knn.predict(X_test)
    rmses_w[i]=root_mean_squared_error(y_test, preds)

plt.plot(ks, rmses, label="uniform")
```

```
plt.plot(ks, rmses_w, color="darkorange", label="distance")
plt.xlabel("K")
plt.ylabel("rmse")
plt.grid()
plt.legend()
plt.show()
```




## 2. Data Scaling

Data scaling is absolutely required for KNN modeling, as the distances are influenced directly by the magnitude of the atributes. Some of the most common scalers for Machine Learning are:

- Standard Scaler - Transforms each column of the dataset into a new feature with Mean = 0 and Variance =1
- MinMax Scaler - Scales each feature such that it fits with a specifed interval (generally [0, 1])
- Power Transformer - Applies a power transformation to each value, such that the resulting feature has the best possible approach to a normal distribution.
- Normalizer - Does not work on columns, but on rows, certifing that the norm of each intance is a unit vector.

We will use the Wine dataset to test these transformations.

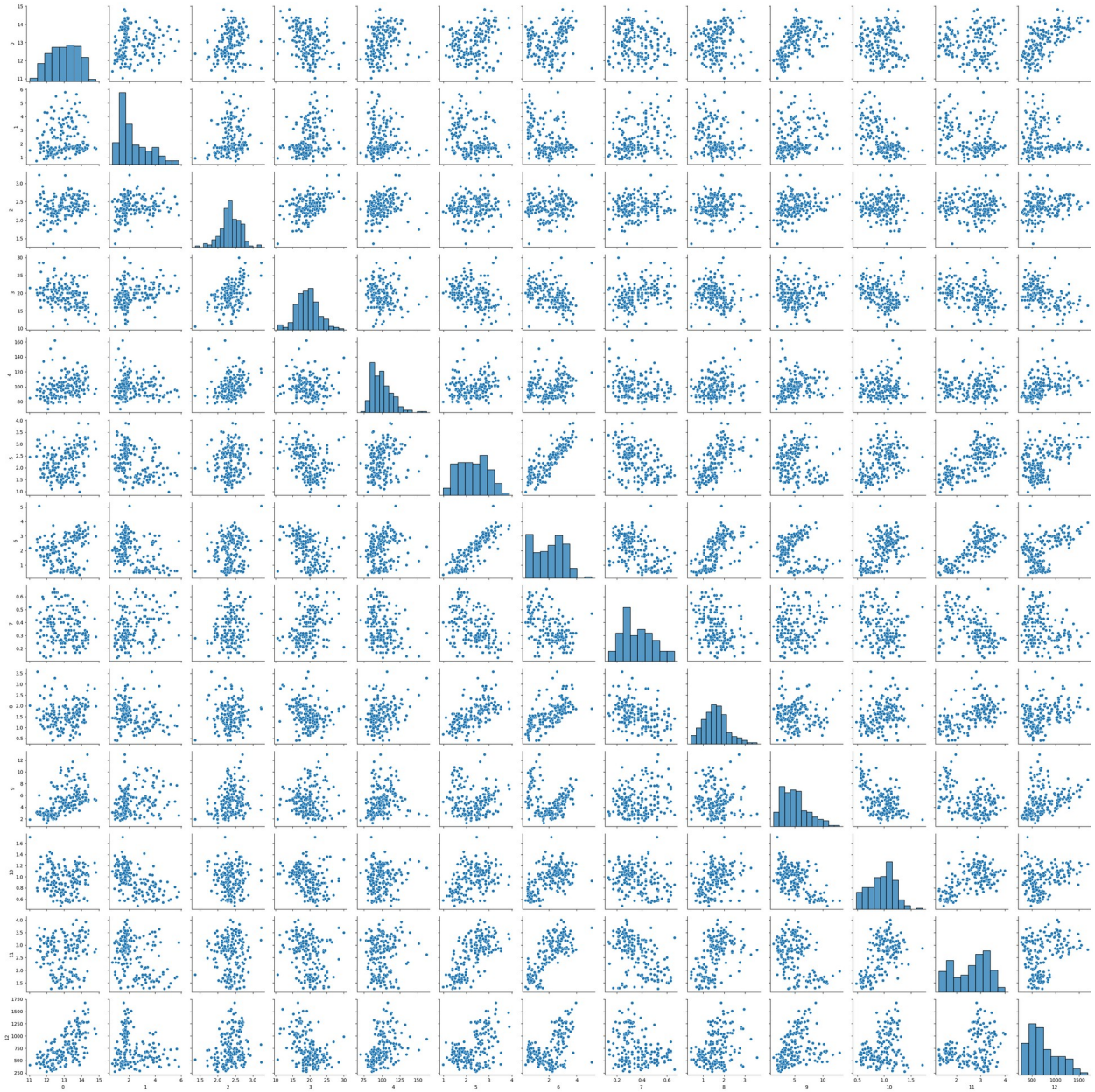
```
from sklearn.datasets import load_wine

X, y = load_wine(return_X_y=True)

# KNN without scaling
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
pred1 = knn.predict(X_test)

print("Accuracy without scaling:", accuracy_score(y_test, pred1))
```

```
Accuracy without scaling: 0.7407407407407407
```

Let's have a look at the data and verify that each feature have a different value range for its values.

```
#let's look at the data
import seaborn as sns
g = sns.PairGrid(pd.DataFrame(X))
g.map_diag(sns.histplot)
g.map_offdiag(sns.scatterplot)
```

```
<seaborn.axisgrid.PairGrid at 0x2857e460ef0>
```



**We have to scale the features values!**

We will use Standardization, which transforms features to have:

- Mean = 0
- Standard deviation = 1

$$z = \frac{x - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Now we have all the features with the same ranges! Let's check it out.

```
g = sns.PairGrid(pd.DataFrame(X_train_scaled))
g.map_diag(sns.histplot)
g.map_offdiag(sns.scatterplot)


knn_scaled = KNeighborsClassifier(n_neighbors=5)

knn_scaled.fit(X_train_scaled, y_train)

pred_scaled = knn_scaled.predict(X_test_scaled)

acc_scaled = accuracy_score(y_test, pred_scaled)

print("Accuracy with scaling:", acc_scaled)
```

## When Is Scaling Necessary?

Scaling is important for algorithms based on distance or magnitude, including:

- K-Nearest Neighbours
- K-Means clustering
- Support Vector Machines
- Principal Component Analysis

Scaling is less important for algorithms based on decision trees, such as:

- Decision Trees
- Random Forests
- Gradient Boosting